⟨Universidad · Facultad · Carrera⟩

Ingeniería de Software 1 (SW1)

SEGUNDO EXAMEN PARCIAL

Diagramador UML

Herramienta CASE colaborativa con Moderador IA para diagramas de clases UML 2.5 y generación de backends Spring Boot

Grupo: 43

Integrantes:

Elizabeth Rivero Peña

Luis Fernando Angulo Heredia

Docente: Ing. Rolando Martínez Canedo

Fecha de entrega: miércoles 11 de noviembre de 2026

Repositorio (GitHub)

Aplicación en producción

Índice

Sección 1. Fundamentación teórica de la aplicación (2do Examen Parcial) 4
1.1 La Ingeniería de Software Asistida por Computadora (CASE) 4
1.2 Aceleración de la productividad: desarrollo basado en componentes y reuso 5
1.3 Arquitectura del software (los 4 tipos) 5
1.4 Propósito del software: “dar larga vida al software” 8
1.5 IA local: diferencia entre entrenar y configurar un modelo 8
1.6 SDD — Specification-Driven Development (Desarrollo Dirigido por Especificaciones) 9
1.7 Estándares UML 10
1.8 Mecanismo de onboarding del usuario (agente de ayuda) 11
Referencias de esta sección (formato APA 7) 11
Sección 2. Registro del desarrollo (evidencia del proceso) 12
2.1 Control de versiones y forma de trabajo 12
2.2 Línea de tiempo del 1er Parcial (base del proyecto) 12
2.3 Registro del 2do Parcial: plan vs. ejecución real 13
2.4 Evidencias de pruebas funcionales 15
2.5 Registro de defectos encontrados y corregidos 16
2.6 Trazabilidad requisito → código → prueba (insumo para V&V) 17
2.7 Herramientas utilizadas y uso de asistencia de IA 18
2.8 Pendientes registrados (al 17/10/2026) 18
Sección 3. Mecanismo de aprendizaje del usuario 19
3.1 Enfoque: la aplicación enseña a usarse a sí misma 19
3.2 Nivel 1 — Estado vacío del lienzo 19
3.3 Nivel 2 — Recorrido guiado (tour) 19
3.4 Nivel 3 — Ayuda bajo demanda 20
3.5 Nivel 4 — Retroalimentación mientras se trabaja 21
3.6 Aprendizaje en el cliente móvil 22
3.7 Cómo aprende un usuario nuevo: recorrido de ejemplo 22
Anexos 23
Anexo A. Evidencia del sistema en funcionamiento 23
Anexo B. Estándar de Codificación 25
Anexo C. Información del grupo y recursos del proyecto 26

Anexo D. Metodología de investigación y de desarrollo 26
Anexo E. Modelos UML 2.5.1 del 2do Parcial 27
Anexo F. Bibliografía (APA 7) 35

Sección 1. Fundamentación teórica de la aplicación (2do Examen Parcial)

1.1 La Ingeniería de Software Asistida por Computadora (CASE)

Una herramienta **CASE** (*Computer-Aided Software Engineering*) es un software que automatiza o asiste actividades del proceso de desarrollo: análisis, diseño, generación de código, documentación, pruebas y gestión. Su objetivo es aumentar la productividad y la calidad, reduciendo los errores manuales que se producen al pasar de una fase a otra (Sommerville, 2016).

Según la parte del ciclo de vida que cubren, las herramientas CASE se clasifican en:

Tipo	Fases que cubre	Ejemplos
Upper CASE (alto nivel)	Requisitos, análisis y diseño (modelado)	Enterprise Architect, StarUML
Lower CASE (bajo nivel)	Implementación, generación de código, pruebas	Generadores de código, compiladores
I-CASE (integradas)	Cubren ambas, con un repositorio común	Rational Rose, herramientas MDA

Beneficios: coherencia entre diseño e implementación, menor tiempo de desarrollo, documentación siempre actualizada, estandarización (UML) y trabajo en equipo sobre un mismo modelo.

Nuestra aplicación es una herramienta I-CASE, porque integra en un mismo repositorio (el diagrama) las dos mitades del ciclo:

- **Upper CASE:** modelado visual de diagramas de clases UML, validación del modelo, interoperabilidad XMI con Enterprise Architect y documentación generada.
- **Lower CASE:** generación automática de un backend **Spring Boot 3.3.4** completo (entidades JPA, repositorios, servicios, controladores REST, Swagger, `schema.sql`, `data.sql`, colección Postman y Docker).
- **Asistencia inteligente (2do Parcial):** un moderador de IA grupal que responde dudas teóricas y modifica el diagrama compartido a pedido.

Dónde se ve en el código

- Modelado: `frontend/src/App.tsx`, `frontend/src/components/UmlClassNode.tsx`, `UmlRelationEdge.tsx`
- Validación del modelo: `backend/src/diagrams/validation.util.ts`
- Interoperabilidad XMI (OMG XMI 2.1): `backend/src/diagrams/xmi.util.ts`, `xmi-import.util.ts`

- Generador: `backend/src/generator/generator.service.ts` y `backend/src/generator/templates/`

1.2 Aceleración de la productividad: desarrollo basado en componentes y reuso

El **Desarrollo Basado en Componentes** (CBSE) construye sistemas ensamblando piezas independientes y reutilizables con interfaces bien definidas, en lugar de escribir todo desde cero. El **reuso** se aplica a varios niveles: código, diseño (patrones) y arquitectura (plantillas de aplicación).

Nuestro generador aplica el reuso de tres formas:

1. **Plantillas como componentes de generación.** Cada capa del backend generado sale de una plantilla independiente y probada: `entity`, `repository`, `service`, `controller`, `enum`, `interface`, `schema-sql`, `data-sql`, `postman`, `openapi-config`, `jackson-config` y `project-files`. Agregar una capa nueva es agregar un componente sin tocar los demás.
2. **Reuso de componentes del ecosistema Spring.** El código generado no reimplementa la persistencia ni la API: se apoya en componentes maduros (Spring Data JPA, Spring Web, springdoc-openapi/Swagger), que es lo que permite que un diagrama se convierta en un backend funcional en segundos.
3. **Arquitectura en capas como plantilla de aplicación.** Todo backend generado repite la misma estructura Controller → Service → Repository → Entity, así que cualquier desarrollador ya sabe dónde está cada cosa.

En el frontend se aplica el mismo principio: la interfaz se compone de **31 componentes React** reutilizables (lienzo, inspector, chat, paneles), y la lógica pura (diferencias, métricas, exportaciones) vive en módulos sin UI.

Dónde se ve en el código

- Plantillas (cada una con su test): `backend/src/generator/templates/*.template.ts` y `*.spec.ts`
- Componentes de UI: `frontend/src/components/` · lógica reutilizable: `frontend/src/lib/`

1.3 Arquitectura del software (los 4 tipos)

1.3.1 Arquitectura de backend — monolito modular por responsabilidades

El backend de la herramienta CASE es un **monolito modular en NestJS** (Node.js + TypeScript). Cada módulo encapsula una responsabilidad y expone solo lo necesario mediante inyección de dependencias:

Módulo	Responsabilidad

Módulo	Responsabilidad
`auth`	Registro, login y JWT; guard global que protege la API REST
`users`	Usuarios (PostgreSQL)
`diagrams`	Diagramas, autoría granular, bitácora, sesiones continuas, gateway de colaboración en tiempo real
`ai`	Motor IA híbrido (Gemini + Ollama), moderador grupal y gateway del chat
`generator`	Generación del backend Spring Boot en ZIP
`attachments`	Archivos adjuntos en Amazon S3

Dentro de cada módulo se respeta la separación **Controller (HTTP) / Gateway (WebSocket) → Service (lógica) → Schema/Entity (datos)**. En el motor de IA se aplica el **patrón Strategy** (`AiProvider`, con dos implementaciones: `GeminiProvider` y `OllamaProvider`) compuesto por `HybridAiProvider`, que agrega fallback automático y *circuit breaker*.

Evidencia: `backend/src/app.module.ts`, `backend/src/*/…module.ts`, `backend/src/ai/providers/`.

1.3.2 Arquitectura de frontend — componentes, hooks y capa de API

- **Arquitectura de componentes (React + Vite + TypeScript):** cada parte de la UI es un componente independiente (`ChatPanel`, `ClassInspector`, `UmlClassNode`, `HistoryPanel`…).
- **Estado global con Context API:** `AuthContext` (sesión y JWT) y `ToastContext` (avisos).
- **Hooks personalizados** para efectos secundarios: `useSpeechRecognition` (dictado por voz).
- **Capa de acceso a datos separada:** `api/client.ts` (REST con Axios e interceptor JWT) y `api/socket.ts` (Socket.io con el JWT en el *handshake*). Los componentes no hablan directamente con el servidor.
- **Lienzo con React Flow**, que representa clases como nodos y relaciones como aristas.

Evidencia: `frontend/src/components/`, `frontend/src/context/`, `frontend/src/hooks/`, `frontend/src/api/`.

1.3.3 Arquitectura de datos — persistencia políglota

En el 2do Parcial adoptamos **persistencia políglota**: cada dato se guarda en el motor que mejor se adapta a su forma.

Motor	Colecciones / tablas	Por qué

Motor	Colecciones / tablas	Por qué
MongoDB (NoSQL documental)	`diagrams`, `diagram_sessions`, `diagram_activity_logs`, `group_chat_messages`	El modelo UML es un **árbol** (clases → atributos, relaciones) con la autoría **embebida** en cada elemento; las sesiones y la bitácora crecen con el uso. Un documento evita *joins* y se guarda tal cual.
PostgreSQL (relacional)	`users`, `attachments`	Datos relacionales con restricciones (correo único), estables y ya probados en el 1er Parcial.

Decisiones clave:

- **Autoría granular:** cada clase, atributo y relación guarda `author`, `lastModifiedBy` y `source` (MANUAL, AI, XMI_IMPORT). La calcula el servidor comparando el modelo anterior con el nuevo, así que **el cliente no puede falsificarla**.
- **Bitácora estructurada:** una entrada por cambio (`CLASS_CREATED`, `ATTRIBUTE_ADDED`, `RELATION_DELETED`…), que permite a la IA responder *"¿qué hizo Marco?"* con datos reales.
- **Migración sin pérdida:** un script idempotente copió los datos del 1er Parcial conservando los mismos identificadores (UUID).

Evidencia: `backend/src/diagrams/schemas/`, `backend/src/diagrams/authorship.util.ts`, `backend/src/scripts/migrate-pg-to-mongo.ts`.

1.3.4 Arquitectura de despliegue — contenedores en la nube

```
Usuario (web / móvil Flutter)
        │ HTTPS (Let's Encrypt)
        ▼
  nginx (contenedor frontend) ── sirve el SPA React
        │  /auth /diagrams /ai /sessions /generator /attachments  → proxy
        │  /socket.io                                              → WebSocket
        ▼
  backend NestJS (contenedor) ──┬── MongoDB 7 (contenedor, volumen mongo_data)
                                ├── PostgreSQL 16 (contenedor, volumen db_data)
                                ├── Amazon S3 (adjuntos)
                                └── Gemini API (IA en la nube) / Ollama (IA local)
```

- **Docker Compose** orquesta cuatro contenedores separados con *healthchecks* y reinicio automático; los datos viven en volúmenes persistentes.
- **Servidor en la nube** (instancia AWS EC2 de 1 GB) con dominio **DuckDNS** (`diagramasw1pracial100.duckdns.org`) y certificado **SSL/TLS de Let's Encrypt**; nginx redirige HTTP → HTTPS.
- **Despliegue por compilación local:** el `dist/` se compila en la PC y se sube por SCP, porque compilar en el servidor de 1 GB agotaba la memoria.

- **Plan B local:** `docker-compose.local.yml` levanta la misma pila en una laptop con IA local, para defender sin internet.

Evidencia: `docker-compose.prod.yml`, `docker-compose.local.yml`, `frontend/nginx.conf`, `deploy/deploy-local-build.ps1`.

1.4 Propósito del software: "dar larga vida al software"

Un software vive mientras pueda **cambiar a bajo costo**. La mayor parte del costo total ocurre después de la primera versión, en mantenimiento y evolución (Sommerville, 2016). Para alargar la vida del sistema aplicamos:

1. **Estándares de programación verificados por herramientas:**
    - TypeScript con tipado estricto (backend con `strict: true`; frontend con `noUnusedLocals`, `noUnusedParameters` y `noFallthroughCasesInSwitch`).
    - Linter **oxlint** (reglas de React Hooks incluidas) y formateador **Prettier**; en Flutter, `flutter_lints`.
    - El estándar de codificación del equipo está documentado en el Anexo B.
2. **Diseño modular y extensible:** módulos con una sola responsabilidad, plantillas independientes en el generador y contratos explícitos (`AiProvider`). Agregar un tercer proveedor de IA o una nueva capa generada no obliga a modificar el resto.
3. **Pruebas automatizadas: 106 pruebas unitarias** (Vitest) cubren el generador, la validación, XMI, la autoría, el motor híbrido y el prompt del moderador. Fallan si un cambio rompe un comportamiento existente.
4. **Evolución sin romper lo anterior:** el 2do Parcial agregó MongoDB, IA grupal y el visor móvil **manteniendo intactas** las funciones del 1er Parcial (verificado con pruebas de regresión: XMI, generador, locks).
5. **Configuración externa:** URL, claves y proveedores de IA se definen con variables de entorno (`.env.example` documentado), no en el código.

Evidencia: `backend/tsconfig.json`, `.prettierrc`, `oxlint.json`, `docs/estandar_de_codificacion.md`, `backend/src/**/*.spec.ts`.

1.5 IA local: diferencia entre entrenar y configurar un modelo

	Entrenar un modelo	**Configurar un modelo (lo que hicimos)**
Qué se hace	Ajustar millones de parámetros con grandes volúmenes de datos	Usar un modelo **ya preentrenado** y guiarlo con instrucciones
Requiere	Datasets, GPU/TPU, días o semanas de cómputo	Un *runtime* (Ollama) y un buen *System Prompt*
Resultado	Un modelo nuevo (pesos nuevos)	El mismo modelo con un **rol** y un

	Entrenar un modelo	Configurar un modelo (lo que hicimos)
		contexto definidos

Nosotros configuramos, no entrenamos. Usamos **qwen2.5:1.5b-instruct** (Alibaba) ejecutado localmente con **Ollama**, y **Gemini** (Google AI Studio) en la nube. Lo configuramos así:

- **System Prompt de rol:** *"Eres el Moderador y Desarrollador Senior del equipo… dirígete siempre por su nombre a quien pregunta… solo modifica el lienzo si te lo piden explícitamente"*.
- **Contexto dual inyectado en cada pregunta:**
  - **Grupal:** el estado del diagrama con su autoría, la bitácora de quién hizo qué y el chat público de la sala.
  - **Individual:** el hilo de preguntas previas de quien pregunta.
- **Ejemplos (few-shot)** y un mini glosario UML para el modelo pequeño.
- **Parámetros de inferencia:** temperatura 0 (respuestas deterministas) y salida en JSON con un contrato fijo de operaciones.
- **Validación determinista:** el servidor descarta operaciones inválidas y las que no responden a un pedido explícito. **La IA nunca toca el diagrama sin control.**

Motor híbrido (local + nube): ambos proveedores comparten el mismo contrato. Por defecto responde Gemini, y **Ollama toma el relevo automáticamente si no hay internet**; con `AI_HYBRID_ORDER=local-first` se invierte el orden para la demostración. Lo medimos en la laptop del equipo (CPU): el prompt completo tardaba 36–79 s en el modelo local; el *prompt lite*, con caché de prefijo, baja a 5–18 s.

Dónde se ve en el código: `backend/src/ai/moderator/moderator-prompt.ts`, `backend/src/ai/providers/hybrid.provider.ts`, `ollama.provider.ts`, `backend/src/ai/moderator/intent.ts`.

1.6 SDD — Specification-Driven Development (Desarrollo Dirigido por Especificaciones)

En SDD la **especificación es el artefacto central**: el código se deriva de ella, y no al revés. En nuestra herramienta, **el diagrama de clases UML es la especificación** y el generador la transforma directamente en un backend ejecutable:

```
Especificación (diagrama UML) ──validación──▶ Generador ──▶ Backend Spring Boot
   clases, atributos, tipos,                  plantillas      entidades JPA, repos,
   relaciones, multiplicidades                                servicios, REST, SQL
```

Se relaciona con el **Proceso Unificado (PUDS)** de Jacobson, Booch y Rumbaugh (1999), que es dirigido por modelos: el modelo de diseño guía la implementación.

- **Ventajas:** el proceso es claro y estructurado, el código es coherente con el diseño, la generación es inmediata y el modelo sirve como documentación viva.
- **Desventaja (la "ventana" del SDD):** *si se olvidó algo en la especificación, todo lo generado lo arrastra.* Una relación mal declarada produce claves foráneas incorrectas en todas las capas.
- **Cómo mitigamos esa ventana:** el validador revisa el modelo **antes** de generar (clases sin nombre o sin clave primaria, clases o atributos duplicados, relaciones hacia clases inexistentes, atributos desnormalizados y claves foráneas escritas a mano), la IA moderadora señala problemas de modelado, y la bitácora con autoría permite rastrear quién introdujo cada elemento.

Dónde se ve en el código: `backend/src/diagrams/validation.util.ts`, `backend/src/generator/`.

1.7 Estándares UML

- **UML** es el lenguaje estándar de modelado de software, mantenido por el **OMG** (*Object Management Group*, omg.org). La versión vigente es **UML 2.5.1** (Object Management Group, 2017).
- Nuestro diagrama de clases soporta la notación UML 2.x:
    - Clases con atributos y operaciones, visibilidad (`+ - # ~`) y estereotipos («`enum`», «`abstract`», «`interface`»).
    - Relaciones de asociación, agregación, composición, herencia y dependencia, con multiplicidades 1:1, 1:N, N:1 y N:M y nombres de rol.
- **Intercambio estándar:** los diagramas se exportan e importan en **XMI** (*XML Metadata Interchange*, también del OMG), compatible con Enterprise Architect.
- **Reglas de modelado — OMT:** la *Object Modeling Technique* de Rumbaugh et al. (1991) es la base del modelo de clases de UML. De sus reglas de modelado salen las que aplica nuestro validador:
    - toda clase tiene nombre único e identifica a sus objetos (clave primaria);
    - una clase no repite atributos;
    - las asociaciones solo unen clases existentes;
    - los vínculos entre clases se modelan como **relaciones** y no como atributos sueltos (por eso se avisa de las claves foráneas escritas a mano y de los atributos desnormalizados).

Dónde se ve en el código: `backend/src/diagrams/uml.types.ts`, `frontend/src/components/UmlLegend.tsx`, `backend/src/diagrams/xmi.util.ts`.

1.8 Mecanismo de onboarding del usuario (agente de ayuda)

La aplicación **enseña a usarse a sí misma**, sin documentación externa:

1. **Recorrido guiado (Tour):** la primera vez que el usuario entra, un tour de 7 pasos resalta cada parte de la interfaz. Los pasos son:
    1. dar nombre al diagrama;
    2. agregar una clase;
    3. elegir el tipo de relación;
    4. validar el modelo;
    5. generar el backend Spring Boot;
    6. exportar a XMI;
    7. usar el asistente de IA.

    Se puede repetir desde el botón **"Recorrido"** de la barra.
2. **Ayuda contextual:**
    - Un **estado vacío** del lienzo con las primeras acciones sugeridas.
    - Una **leyenda UML** que explica la notación.
    - Un modal de **atajos de teclado**.
    - Una **paleta de comandos** (`Ctrl+K`).
    - **Plantillas** de diagramas de ejemplo.
3. **Agente de ayuda en línea (IA):** el Moderador IA del chat público responde dudas sobre cómo modelar y sobre conceptos de ingeniería de software, dirigiéndose por su nombre a quien pregunta. Como el chat es público, lo que aprende un integrante queda visible para todo el equipo.

Dónde se ve en el código: `frontend/src/components/Tour.tsx` (pasos en `TOUR_STEPS`, `App.tsx`), `CanvasEmptyState.tsx`, `UmlLegend.tsx`, `ShortcutsModal.tsx`, `CommandPaletteModal.tsx`, `TemplatesPanel.tsx`, `ChatPanel.tsx`.

Referencias de esta sección (formato APA 7)

Jacobson, I., Booch, G., & Rumbaugh, J. (1999). *The unified software development process*. Addison-Wesley.

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

Rumbaugh, J., Blaha, M., Premerlani, W., Eddy, F., & Lorensen, W. (1991). *Object-oriented modeling and design*. Prentice Hall.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Sección 2. Registro del desarrollo (evidencia del proceso)

2.1 Control de versiones y forma de trabajo

- **Repositorio:** `github.com/luisfernandoAngulo28/generador_diagrama_clases_uml` (Git + GitHub), con historial completo desde el inicio del proyecto.
- **Volumen: 89 commits** entre el **30/08/2026** y el **17/10/2026** (hasta `b014a42`; 79 del 1er Parcial y 10 del 2do).
- **Ramas:**
  - El 1er Parcial se desarrolló sobre `main`.
  - El 2do Parcial se aisló en la rama **`feat/mongodb-autoria`** para no romper lo ya evaluado.
  - Esa rama se integra mediante el **Pull Request #1**, que GitHub reporta sin conflictos con `main`.
- **Convención de mensajes:** desde septiembre se usa *Conventional Commits* (`feat:`, `fix:`, `perf:`, `docs:`). Cada commit del 2do Parcial explica **qué** cambió y **por qué**.
- **Planificación:** `PLAN_EVOLUCION_2DO_PARCIAL.md` y `CONTEXTO_CLAUDE_2DO_PARCIAL.md` (cronograma de 4 semanas y checklist), y `PLAN_SEMANA1_MONGODB.md` (diseño de la migración antes de escribir código).

[Insertar captura1: historial de commits en GitHub. Captura 2: el PR #1 con “No conflicts with base branch”.]

2.2 Línea de tiempo del 1er Parcial (base del proyecto)

Semana (ISO)	Fechas	Commits	Funcionalidades entregadas
W35	30 ago	8	Lienzo UML, generador Spring Boot, chat IA, colaboración en tiempo real (Socket.io), dictado por voz, foto de pizarra a diagrama, exportación XMI, plantilla Flutter
W36	31 ago – 06 sep	56	Documentación PUDS con 28 diagramas, validación y normalización del modelo, despliegue en AWS (Docker Compose),

Semana (ISO)	Fechas	Commits	Funcionalidades entregadas
			tests unitarios, tour de onboarding, estereotipos «enum», «abstract» e «interface», bloqueo por clase, deshacer/rehacer, plantillas tipo EA, autenticación JWT, notación UML 2.5, bitácora de cambios, adjuntos en S3, HTTPS con Let's Encrypt en DuckDNS, Swagger y Docker en el ZIP generado
W37–W38	12–20 sep	4	Plan B local: pila completa sin AWS e IA local con Ollama; reintento ante errores 503 de Gemini
W39	21–23 sep	11	Verificación de IA offline en dispositivo físico, estándar de codificación, métricas OO, diagramas de secuencia, BDD, mock API, gestión de usuarios, importación del XMI de Enterprise Architect

2.3 Registro del 2do Parcial: plan vs. ejecución real

El cronograma planificado distribuía el trabajo en cuatro semanas (17/10 al 11/11). En la práctica, las cuatro etapas se implementaron y verificaron el **17/10/2026**, como muestran las fechas de los commits. El tiempo restante hasta la entrega queda para la documentación, el despliegue y el simulacro.

Etapa del plan	Commit	Fecha	Funcionalidad implementada	Cómo se verificó
Preparación	`3ede0cd`	17/10	Plan del 2do Parcial, diagramas de actores y EMCU por ciclo	Revisión del plan
Semana 1 — MongoDB, autoría y sesiones	`cfdd410`	17/10	Persistencia políglota (Mongo + PostgreSQL), autoría granular por	7 tests nuevos de autoría; migración real de 8 diagramas y 55 entradas de

Etapa del plan	Commit	Fecha	Funcionalidad implementada	Cómo se verificó
			clase, atributo y relación, bitácora estructurada, sesiones continuas, JWT en Socket.io, script de migración	historial (idempotente); prueba E2E de las funciones del 1er Parcial: XMI de ida y vuelta, ZIP del generador, locks
Semana 1 (ajuste)	`803ab14`	17/10	nginx enruta `/sessions`	Pila Docker completa probada detrás de nginx
Semana 2 — motor IA híbrido	`914167e`	17/10	`HybridAiProvider` (Ollama ↔ Gemini), System Prompt del moderador con contexto dual, `POST /ai/moderator/:id`	12 tests (motor y prompt); preguntas reales teórica, de contexto, de seguimiento y de acción
Semana 2 (configuración)	`a53a264`	17/10	`AI_PROVIDER=hybrid` documentado	—
Semana 2 (rendimiento)	`679ed16`	17/10	Prompt lite para el modelo local; filtros deterministas	Medición de latencia en CPU (ver 2.4.2); 14 tests nuevos
Semana 2 (orden configurable)	`602a785`	17/10	`AI_HYBRID_ORDER` cloud-first/local-first; precalentamiento de Ollama	4 tests nuevos; tres escenarios reales (ver 2.4.2)
Semana 3 — chat grupal	`814d912`	17/10	`chat:send` / `chat:new-message` / `ai:status`, ChatPanel público con badge del proveedor, autoría en vivo en el inspector	Prueba en navegador con dos usuarios simultáneos (ver 2.4.3)
Semana 4 — visor Flutter	`e5eeae3`	17/10	`UmlViewerScreen` de solo lectura con chat y notificaciones locales	`flutter analyze` sin problemas; versión web probada contra el backend; `flutter build apk --release` exitoso

Etapa del plan	Commit	Fecha	Funcionalidad implementada	Cómo se verificó
Documentación	`b014a42`	17/10	Sección 1 del SDD (fundamentación teórica)	Cada afirmación referida a un archivo del código

Tamaño del cambio del 2do Parcial: 94 archivos, **+8.673 / −252 líneas** (`348f1aa..b014a42`).

2.4 Evidencias de pruebas funcionales

2.4.1 Pruebas automatizadas

- **Backend: 106 pruebas unitarias en 19 archivos, todas aprobadas** (`npx vitest run`, 17/10/2026). Cubren el generador y sus plantillas, validación, XMI, autoría y bitácora (`authorship.util.spec.ts`), saneamiento del modelo, motor híbrido (`hybrid.provider.spec.ts`), prompt del moderador, detección de pedidos explícitos y validación de operaciones de la IA.
- **Frontend:** compilación TypeScript y build de Vite sin errores; linter sin advertencias nuevas.
- **Flutter:** `flutter analyze` sin problemas en el código nuevo; APK de release compilado.

[Insertar captura3: salida de `npx vitest run` con “106 passed”.]

2.4.2 Motor de IA híbrido — mediciones reales (laptop del equipo, CPU)

Escenario	Proveedor que respondió	Tiempo
`cloud-first` (por defecto)	Gemini Cloud	4,3 s
Gemini falla → respaldo automático	Ollama Local	13,5 s
`local-first`, modelo precalentado	Ollama Local	5,0 s
Modelo local con el prompt completo (antes de optimizar)	Ollama Local	36–79 s
Modelo local con prompt lite + caché (después)	Ollama Local	5–18 s

Comprobaciones de comportamiento del moderador:

- **Nombra a quien pregunta:** *“Trevor, un atributo derivado es…”*.
- **Contexto grupal:** respondió *“la clase Pedido fue creada por Marco…”* leyendo la bitácora.
- **Contexto individual:** *“como te comenté antes…”*.

- **Acción solo a pedido:** ante *“Agrega una clase Producto… muchos a muchos con Pedido”* propuso `CREATE_CLASS` + `MANY_TO_MANY`. Ante preguntas, cero operaciones.

[Insertar captura4: log del backend con los tiempos de Ollama. Captura 5: badges 🏠 Ollama Local / ☁ Gemini Cloud en el chat.]

2.4.3 Prueba de colaboración con dos usuarios (navegador)

Dos sesiones simultáneas (Marco y Luis) sobre el mismo diagrama:

1. Luis pide a la IA crear `Producto` con relación N:M a `Pedido`.
2. La pestaña de Marco muestra en vivo *“La IA está pensando la respuesta para Luis”*, la respuesta *“Moderador IA → Luis”* y la clase nueva en su lienzo.
3. El inspector de Marco muestra *“Creada por Luis · vía IA”*.
4. La bitácora registra *“Luis: Creó la clase Producto (vía IA)”*, sin entradas falsas.
5. La barra indica **“2 conectados”**.

[Insertar captura6: chat público con la respuesta de la IA. Captura 7: inspector con la autoría “vía IA”.]

2.4.4 Visor móvil en tiempo real

Con el visor Flutter abierto en el diagrama, Marco pidió desde la web *“Agrega el atributo fecha de tipo LocalDate a Pedido”*. **Sin recargar**, el móvil mostró:

- el atributo nuevo;
- el chat actualizado (de 2 a 4 mensajes);
- *“2 en la sala”*;
- en la hoja de detalle de `Producto`, *“Creada por: Luis · vía IA”*.

[Insertar captura8: visor móvil con el diagrama y la pestaña de chat.]

2.5 Registro de defectos encontrados y corregidos

Las pruebas con dos usuarios reales encontraron defectos que las pruebas unitarias no veían. Este registro es evidencia de un proceso de verificación efectivo.

#	Defecto	Causa raíz	Corrección	Commit
1	Los mensajes WebSocket fallaban al unirse a la sala	El guard JWT global también interceptaba Socket.io (sin cabeceras HTTP)	El guard solo valida HTTP; la identidad del socket se toma del *handshake*	`cfdd410`
2	La ruta `/sessions` daba 404 en producción	nginx no la enrutaba al backend	Se agregó a la regla del proxy	`803ab14`

#	Defecto	Causa raíz	Corrección	Commit
3	Respuesta de Ollama en 36–79 s	Prompt de 1.380 tokens en CPU (~50 tok/s)	Prompt lite + caché de prefijo + precalentamiento	`679ed16`, `602a785`
4	La IA local creaba clases que nadie pidió	El ejemplo few-shot de edición se colaba en el moderador	Ejemplos propios + filtro determinista de pedidos explícitos	`679ed16`
5	Una pestaña en segundo plano dejaba de sincronizar	El flag remoto se reseteaba con `requestAnimationFrame`, que no se ejecuta en pestañas ocultas	El flag se consume en el propio efecto de difusión	`814d912`
6	Las clases perdían su posición al recargar (defecto del 1er Parcial, desde `4198c8b`)	`PositionDto` sin decoradores: el `ValidationPipe` borraba x/y	Decoradores de validación + saneamiento del modelo	`814d912`
7	Entradas falsas en la bitácora (“modificó estereotipo…”)	Mongo guardaba `undefined` como `null`	`sanitizeModel` + `ignoreUndefined`	`814d912`
8	El botón del visor quedaba cortado en celulares de 375 px	La AppBar no tenía espacio	Botón flotante “Visor UML en vivo”	`e5eeae3`

2.6 Trazabilidad requisito → código → prueba (insumo para V&V)

Requisito del 2do Parcial	Implementación	Verificación
La IA nombra a quien responde	`moderator-prompt.ts` (regla 1) + `ensureAddressed()`	`moderator-prompt.spec.ts`; respuestas reales
Chat completamente público	`chat.gateway.ts` (difunde a toda la sala)	Prueba con dos usuarios
Mentor teórico y práctico	Prompt del moderador + operaciones + `intent.ts`	`intent.spec.ts`, `validate-operations.spec.ts`
Contexto grupal e individual	`moderator.service.ts` (bitácora, sala, hilo)	Respuestas “creada por Marco” / “como te comenté antes”
IA híbrida local + nube	`hybrid.provider.ts`	`hybrid.provider.spec.ts`; tres escenarios medidos
Autoría granular	`authorship.util.ts`	`authorship.util.spec.ts`; inspector y visor

Requisito del 2do Parcial	Implementación	Verificación
Sesiones continuas	`sessions.service.ts`	E2E: pausa y reanudación con el mismo ID
Visor móvil en tiempo real	`flutter-template/lib/live/`	Prueba en vivo web → móvil; APK compilado
No romper el 1er Parcial	Rama aislada + interfaz de `DiagramsService` intacta	E2E de XMI, generador y locks

2.7 Herramientas utilizadas y uso de asistencia de IA

- **Desarrollo:** Git y GitHub, Node.js 24, NestJS 12, React + Vite, Flutter 3.38, Docker Desktop, MongoDB 7, PostgreSQL 16, Ollama 0.40.
- **Asistente de programación:** el equipo trabajó con **Claude Code** (Anthropic) como asistente de programación. Por transparencia, los commits en que participó llevan la línea `Co-Authored-By: Claude` (82 de 89 commits). El equipo definió los requisitos y el plan, tomó las decisiones de diseño (persistencia políglota, orden del motor híbrido, alcance de cada semana), ejecutó las pruebas y revisó cada cambio antes de integrarlo.
- Todos los commits se hicieron desde la cuenta `luisfernandoAngulo28`.

2.8 Pendientes registrados (al 17/10/2026)

- ☐ Desplegar en el VPS: levantar Mongo y correr una sola vez `migrate-pg-to-mongo.js`.
- ☐ Probar las notificaciones en un Android real (en web no se pudo conceder el permiso).
- ☐ Probar el corte de internet real (WiFi apagado) con `cloud-first`.
- ☐ Limitación conocida: el guardado reemplaza el modelo completo (*last-write-wins*), mitigado con locks por clase.
- ☐ Simulacro final con tres usuarios + visor móvil.

Sección 3. Mecanismo de aprendizaje del usuario

3.1 Enfoque: la aplicación enseña a usarse a sí misma

El aprendizaje del usuario se diseñó en **cuatro niveles**, del más guiado al más autónomo. Un usuario nuevo los recorre en ese orden:

Nivel	Mecanismo	Cuándo aparece	Qué aprende el usuario
1. Primer contacto	Estado vacío del lienzo	Al abrir un diagrama sin clases	Por dónde empezar
2. Guiado	Recorrido (tour) de 7 pasos	Automáticamente la primera vez; luego con el botón **Recorrido**	El flujo completo: modelar → validar → generar → exportar → consultar a la IA
3. Consulta	Leyenda UML, atajos, paleta de comandos, plantillas, tooltips	Cuando el usuario los necesita	La notación UML y las funciones avanzadas
4. Retroalimentación	Validador del modelo y Moderador IA del chat	Mientras trabaja	Qué hizo mal y por qué; conceptos de ingeniería de software

3.2 Nivel 1 — Estado vacío del lienzo

Un lienzo en blanco no le dice nada a un principiante. Por eso, cuando el diagrama no tiene clases, se muestra una tarjeta *"Diseña tu Arquitectura de Software"* con cuatro puntos de partida:

1. **Crear primera clase:** agrega una entidad al lienzo.
2. **Cargar plantilla:** abre la galería de plantillas.
3. **Escanear boceto:** reconoce con IA un diagrama dibujado en papel o pizarra.
4. **Atajos de teclado:** muestra los atajos principales.

Implementación: `frontend/src/components/CanvasEmptyState.tsx`

[Insertar captura9: estado vacío del lienzo.]

3.3 Nivel 2 — Recorrido guiado (tour)

La primera vez que el usuario entra, la aplicación inicia sola un **tour de 7 pasos**. Cada paso **resalta el control real de la interfaz** y explica para qué sirve:

Paso	Control resaltado	Lo que explica
1	Nombre del diagrama	Con qué nombre se guarda el diagrama
2	Botón **+ Clase**	Crear una clase y editar sus

Paso	Control resaltado	Lo que explica
		atributos y su clave primaria
3	Selector de relación	Elegir el tipo de relación y arrastrar entre clases para conectarlas
4	**Validar**	Revisar integridad y normalización (3FN) antes de generar
5	**Spring Boot**	Descargar el backend completo de 4 capas
6	Exportar **XMI**	Llevar el modelo a Enterprise Architect (XMI 2.1)
7	**Chat del equipo**	El chat es público; el Moderador IA responde por nombre y solo cambia el diagrama si se lo piden

Detalles de diseño:

- **Se muestra una sola vez:** al terminar se guarda la marca `case-tool-tour-seen` en el navegador.
- **Se puede repetir:** el usuario lo vuelve a lanzar con el botón **Recorrido** de la barra de herramientas.
- **El globo de ayuda se reubica solo** (arriba, abajo o al costado del control) para no salirse de la pantalla.
- **2do Parcial:** el paso 7 se actualizó para presentar el **chat grupal público** en lugar del asistente privado del 1er Parcial.

Implementación: `frontend/src/components/Tour.tsx` (componente); `TOUR_STEPS` y `TOUR_SEEN_KEY` en `frontend/src/App.tsx`.

[Insertar captura10: un paso del tour resaltando el botón “+ Clase”.]

3.4 Nivel 3 — Ayuda bajo demanda

- **Leyenda de notación UML 2.5** sobre el lienzo: muestra qué significa cada flecha y rombo (herencia, composición, agregación, dependencia…), para aprender la notación mientras se modela. `UmlLegend.tsx`
- **Guía de atajos de teclado** (15 atajos): `Ctrl+K`, `Ctrl+Z` / `Ctrl+Y`, `Ctrl+F`, `Supr`, doble clic para editar, arrastrar para relacionar, etc. `ShortcutsModal.tsx`
- **Paleta de comandos** (`Ctrl+K`, estilo VS Code): busca cualquier acción o clase escribiendo su nombre, sin conocer de antemano dónde está el menú. `CommandPaletteModal.tsx`
- **Galería de plantillas** (7): Herencia básica, Composición (todo-parte), Muchos a muchos, Asociación 1:N, Sistema de Restaurante, Sistema Clínico / Hospital y Sistema

Académico Universitario. **Se aprende viendo ejemplos correctos** de cada tipo de relación. `TemplatesPanel.tsx`, `lib/templates.ts`

- **Tooltips descriptivos** en los botones de la barra, por ejemplo *“Deshacer (Ctrl+Z)”* o *“Abrir paleta rápida de comandos (Ctrl + K)”*.

[Insertar captura11: leyenda UML y guía de atajos.]

3.5 Nivel 4 — Retroalimentación mientras se trabaja

3.5.1 El validador como maestro de modelado

Validar no solo dice “error”: explica **qué regla de modelado** se rompió. Estos son los mensajes reales:

- *“La clase”Pedido” no tiene ningún atributo marcado como clave primaria (PK).”*
- *“Hay 2 clases con el mismo nombre (”Cliente”). Los nombres de clase deben ser únicos.”*
- *“Hay una relación que apunta a una clase que ya no existe en el diagrama.”*
- Avisos de **normalización**: atributos desnormalizados y claves foráneas escritas a mano que deberían modelarse como relación.

Así el usuario aprende las reglas de OMT y de normalización **en el momento en que las necesita**. `backend/src/diagrams/validation.util.ts`, `ValidationPanel.tsx`

3.5.2 El Moderador IA: un agente de ayuda en línea (2do Parcial)

El chat público del equipo funciona como un **docente en el aula**:

- **Responde por nombre** a quien pregunta (*“Trevor, aquí tu respuesta…”*), y todos los integrantes ven la pregunta y la respuesta. **Lo que aprende uno lo aprende el equipo.**
- **Mentor teórico:** explica conceptos (atributo derivado, composición vs. agregación, normalización, JPA), con ejemplos tomados **del diagrama del propio equipo**.
- **Mentor práctico:** si se le pide explícitamente (*“agrega la clase Producto y relaciónala muchos a muchos con Pedido”*), aplica el cambio y lo deja marcado como **“vía IA”** en la autoría.
- **Memoria individual:** recuerda las preguntas previas de cada integrante y da continuidad (*“como te comenté antes…”*).
- **Funciona sin internet:** si Gemini no está disponible, responde el modelo local (Ollama).
- **Voz:** también se le puede preguntar por dictado.

`ChatPanel.tsx`, `backend/src/ai/moderator/`, `backend/src/ai/chat/chat.gateway.ts`

[Insertar captura12: el Moderador IA respondiendo una pregunta teórica por nombre.]

3.6 Aprendizaje en el cliente móvil

El visor UML de Flutter está pensado para usarse sin instrucciones:

- **Acceso directo:** el botón flotante **“Visor UML en vivo”** en la pantalla principal.
- **Mismo login de la web:** el usuario entra con la misma cuenta y elige un diagrama de la lista.
- **Texto de ayuda en el lienzo:** *“Solo lectura · pellizca para hacer zoom · toca una clase para ver su autoría”*.
- **Notificaciones:** avisan de cambios y de respuestas de la IA, así el integrante que sigue la clase desde el celular se entera sin buscar.

`flutter-template/lib/live/uml_viewer_screen.dart`

3.7 Cómo aprende un usuario nuevo: recorrido de ejemplo

1. **Entra por primera vez:** el tour lo lleva en 7 pasos por todo el flujo.
2. **Ve el lienzo vacío:** elige **Cargar plantilla** → **Composición** y observa un modelo correcto.
3. **Crea su propia clase y valida:** el validador le avisa *“no tiene clave primaria”* y aprende la regla.
4. **Duda qué relación usar:** consulta la **leyenda UML** o le pregunta al **Moderador IA** en el chat, y su equipo ve la explicación.
5. **Genera el backend Spring Boot.** Nunca abrió un manual.

El **Manual de Usuario** (`MANUAL_DE_USUARIO.md`) existe como respaldo escrito, pero no es necesario para empezar a usar la herramienta.

Anexos

Anexo A. Evidencia del sistema en funcionamiento

Este anexo reúne la evidencia de que el sistema fue construido, desplegado y verificado de extremo a extremo. Las secciones A.1 a A.7 corresponden al 1er Parcial (se mantienen vigentes) y las A.8 a A.12 a lo agregado en el 2do Parcial.

A.1 Despliegue en producción

La herramienta está desplegada en un servidor accesible públicamente, con certificado SSL válido (HTTPS):

- **URL de producción:** `https://diagramasw1pracial100.duckdns.org`
- **Infraestructura:** servidor AWS EC2 con Docker Compose. Contenedores:
    - frontend (nginx con SSL);
    - backend NestJS;
    - PostgreSQL 16;
    - **MongoDB 7** (nuevo en el 2do Parcial).

  Los servicios se reinician solos y HTTP redirige a HTTPS.

  ⟨Al 17/10/2026, la versión del 2do Parcial con MongoDB está probada en local y en Docker, pero aún no desplegada en el VPS. Tomar la Figura A.1 después del despliegue.⟩
- **Dominio y certificado:** DuckDNS + Let’s Encrypt.
- **Almacenamiento en la nube:** los archivos adjuntos de los diagramas se guardan en un *bucket* de Amazon S3.

[Insertar figuraA.1: sitio en producción con el candado de seguridad (HTTPS).]

A.2 Modelado y generación de código

[Insertar figuraA.2: lienzo principal con un diagrama de clases.]

[Insertar figuraA.3: edición de atributos y operaciones de una clase.]

[Insertar figuraA.4: relaciones con multiplicidades en notación UML 2.5.]

[Insertar figuraA.5: código Java generado y Swagger UI con los endpoints CRUD.]

A.3 Colaboración en tiempo real

[Insertar figuraA.6: dos navegadores editando el mismo diagrama, con la presencia de usuarios y el bloqueo por clase.]

A.4 Asistencia por inteligencia artificial (1er Parcial)

[Insertar figuraA.7: generación de clases por chat e interpretación de una fotografía de pizarra.]

A.5 Interoperabilidad con Enterprise Architect (XMI)

[Insertar figuraA.8: modelo exportado e importado en formato XMI dentro de Enterprise Architect.]

A.6 Cliente móvil y IA local verificados en dispositivo físico

- **APK compilado (modo release):** `ExamenSW1.apk`.
- Verificado en un teléfono Android real:
    - creación y listado de registros contra el backend generado (GET y POST persistidos en PostgreSQL);
    - dictado por voz;
    - **asistente de IA local funcionando en modo avión**.

[Insertar figuraA.9: aplicación móvil en funcionamiento.]

A.7 Pruebas automatizadas

El backend tiene **106 pruebas unitarias en 19 archivos** (Vitest), todas aprobadas al 17/10/2026. Cubren:

- el generador y cada plantilla;
- validación y XMI;
- autoría y bitácora;
- el motor de IA híbrido y el prompt del moderador.

[Insertar figuraA.10: ejecución de `npx vitest run` con “106 passed”.]

A.8 Persistencia en MongoDB con autoría granular (2do Parcial)

- Colecciones `diagrams`, `diagram_sessions`, `diagram_activity_logs` y `group_chat_messages`.
- Cada clase, atributo y relación registra quién la creó, quién la modificó y desde qué fuente (manual, IA o XMI).
- Migración desde PostgreSQL con IDs conservados: 8 diagramas y 55 entradas de historial. Se ejecutó dos veces sin duplicar datos.

[Insertar figuraA.11: documento de un diagrama en MongoDB con los campos `author` y `lastModifiedBy`.]

[Insertar figuraA.12: panel de historial con la bitácora por cambio, incluido “(vía IA)”.]

A.9 Chat grupal público con el Moderador IA (2do Parcial)

- Todos los conectados ven cada pregunta y respuesta; la IA nombra a quien pregunta y muestra qué proveedor respondió (🏠 Ollama Local / ☁ Gemini Cloud).
- Prueba con dos usuarios simultáneos:
  - indicador *"La IA está pensando la respuesta para Luis"*;
  - cambio aplicado y sincronizado en ambos lienzos;
  - autoría *"Creada por Luis · vía IA"*.

[Insertar figuraA.13: chat público con una respuesta "Moderador IA → Luis".]

[Insertar figuraA.14: inspector de clase con la sección de autoría.]

A.10 Motor de IA híbrido (2do Parcial)

Mediciones reales en la laptop del equipo (CPU):

Escenario	Respondió	Tiempo
Gemini primero (por defecto)	Gemini Cloud	4,3 s
Falla Gemini → respaldo automático	Ollama Local	13,5 s
Ollama primero, modelo precalentado	Ollama Local	5,0 s

[Insertar figuraA.15: log del backend con el desglose de tokens y tiempos de Ollama.]

A.11 Visor UML en tiempo real en Flutter (2do Parcial)

- El móvil ve en vivo (solo lectura) el diagrama y el chat del equipo; un cambio pedido a la IA desde la web aparece sin recargar.
- Toca una clase para ver su autoría.
- Notificaciones locales de mensajes y cambios.
- APK de release compilado con los cambios.

[Insertar figuraA.16: visor móvil, pestaña Diagrama. Figura A.17: pestaña Chat.]

A.12 Mecanismo de aprendizaje del usuario

[Insertar figuraA.18: tour guiado. Figura A.19: estado vacío del lienzo. Figura A.20: leyenda UML y guía de atajos.]

Anexo B. Estándar de Codificación

El estándar de codificación del proyecto se documenta completo en el anexo correspondiente (`docs/estandar_de_codificacion.md`). Incluye:

- convenciones de nomenclatura y estándares internacionales;
- stack tecnológico;

- herramientas de calidad: oxlint, Prettier, TypeScript estricto, Vitest y flutter_lints;
- configuración del entorno y métricas de calidad.

(Insertar aquí el contenido del documento “Estándar de Codificación”.)

Anexo C. Información del grupo y recursos del proyecto

Campo	Valor
Documento	**Segundo Examen Parcial**
Materia	Ingeniería de Software 1 (SW1)
Docente	Ing. Rolando Martínez Canedo
Grupo	**43**
Integrantes	**Elizabeth Rivero Peña** (registro ⟨…⟩) · **Luis Fernando Angulo Heredia** (registro ⟨…⟩)
Fecha de entrega	Miércoles 11 de noviembre de 2026, 08:00
Repositorio (QR 1)	`https://github.com/luisfernandoAngulo28/generador_diagrama_clases_uml`
Aplicación en producción (QR 2)	`https://diagramasw1pracial100.duckdns.org`
Aplicación móvil	`ExamenSW1.apk` (incluye el Visor UML en vivo)

Anexo D. Metodología de investigación y de desarrollo

D.1 Metodología de desarrollo: PUDS (Proceso Unificado, versión simplificada)

Seguimos el **Proceso Unificado de Desarrollo de Software** (Jacobson, Booch y Rumbaugh, 1999). Sus tres principios se ven en el proyecto:

- **Dirigido por casos de uso:** los requisitos se modelaron como actores y casos de uso por ciclo (`docs/diagrams/actores.*`, `emcu_ciclo1..3.*`).
- **Centrado en la arquitectura:** las cuatro arquitecturas (backend, frontend, datos y despliegue) se definieron antes de construir; por ejemplo, `PLAN_SEMANA1_MONGODB.md` diseñó la persistencia antes de escribir código.
- **Iterativo e incremental:** el 1er Parcial fue la base y el 2do Parcial se construyó en cuatro iteraciones, cada una con su commit y su verificación.

Fase PUDS	Qué se hizo en el 2do Parcial
Inicio	Requisitos del docente, actores y EMCU, plan de 4 semanas (`PLAN_EVOLUCION_2DO_PARCIAL.md`)
Elaboración	Arquitectura de datos políglota, protocolo WebSocket, diseño del motor IA híbrido

Fase PUDS	Qué se hizo en el 2do Parcial
Construcción	Iteración 1: MongoDB y autoría · 2: IA híbrida · 3: chat grupal · 4: visor Flutter
Transición	Pruebas de regresión, documentación SDD, despliegue en el VPS y simulacro

D.2 Métodos de investigación empleados

- **Investigación documental:** especificaciones del OMG (UML 2.5.1, XMI 2.1), bibliografía de ingeniería de software y documentación técnica oficial (NestJS, MongoDB, Socket.IO, Ollama, Gemini).
- **Investigación experimental:** mediciones controladas para decidir el diseño. Por ejemplo, la latencia del modelo local con el prompt completo (36–79 s) frente al prompt lite con caché (5–18 s) determinó que Gemini responda primero por defecto y Ollama quede como respaldo sin internet.
- **Prueba y verificación:** pruebas unitarias automatizadas, pruebas de punta a punta y pruebas con varios usuarios simultáneos. Los defectos encontrados y sus causas se registran en la Sección 2.5.

Anexo E. Modelos UML 2.5.1 del 2do Parcial

Los siguientes modelos usan notación **UML 2.5.1** (Object Management Group, 2017) y se construyeron a partir del código real. Debajo de cada figura se indica qué archivos del código refleja: es la evidencia de V&V de que *lo que está en el código está reflejado en los modelos*. Las fuentes PlantUML están en `docs/diagrams/2do_parcial/`.

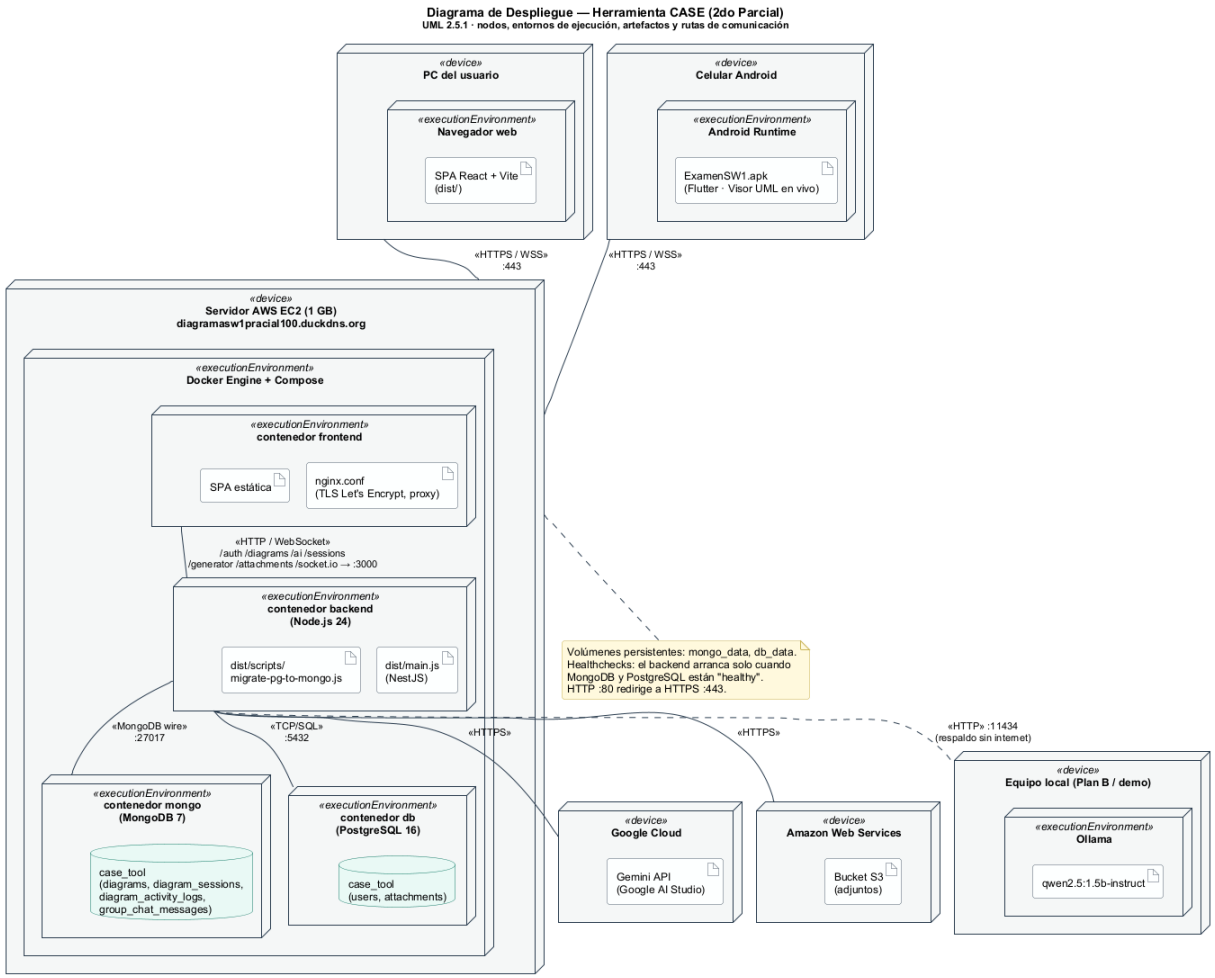


Figura E.1. Diagrama de despliegue (nodos, entornos de ejecución, artefactos y rutas de comunicación)

Refleja el código en: `docker-compose.prod.yml`, `docker-compose.local.yml`, `frontend/nginx.conf`

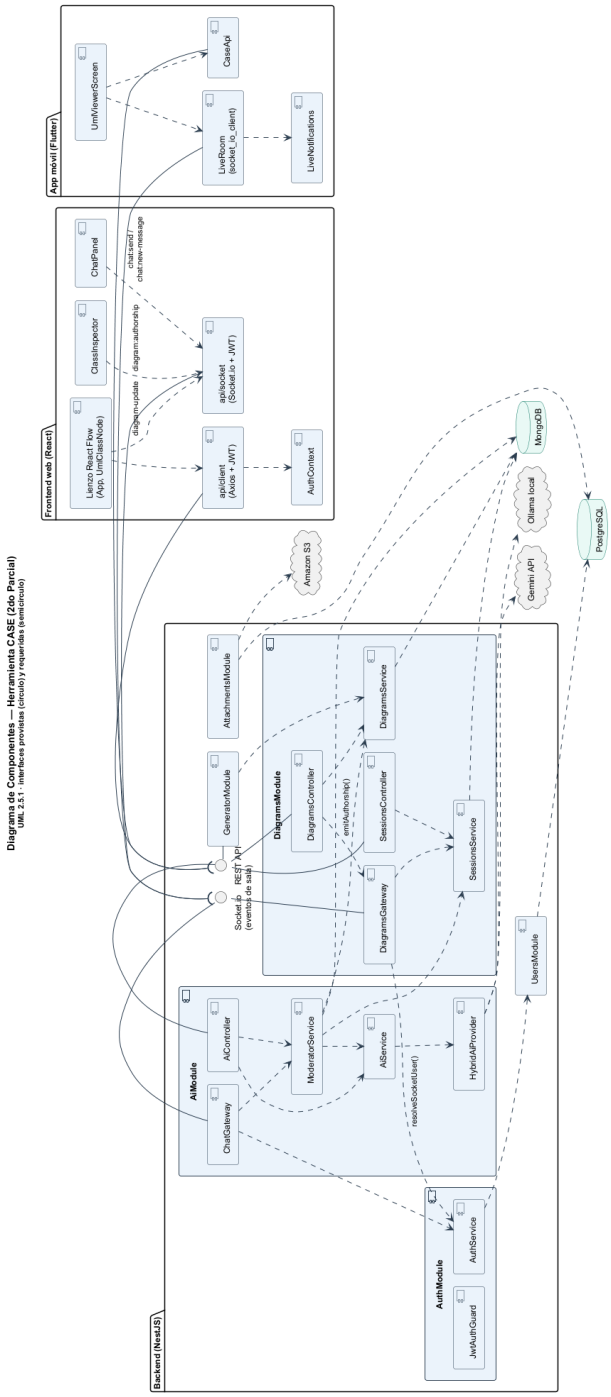


Figura E.2. Diagrama de componentes (interfaces REST y Socket.io provistas y requeridas)

Refleja el código en: `backend/src/*/…module.ts, frontend/src/api, flutter-template/lib/live`

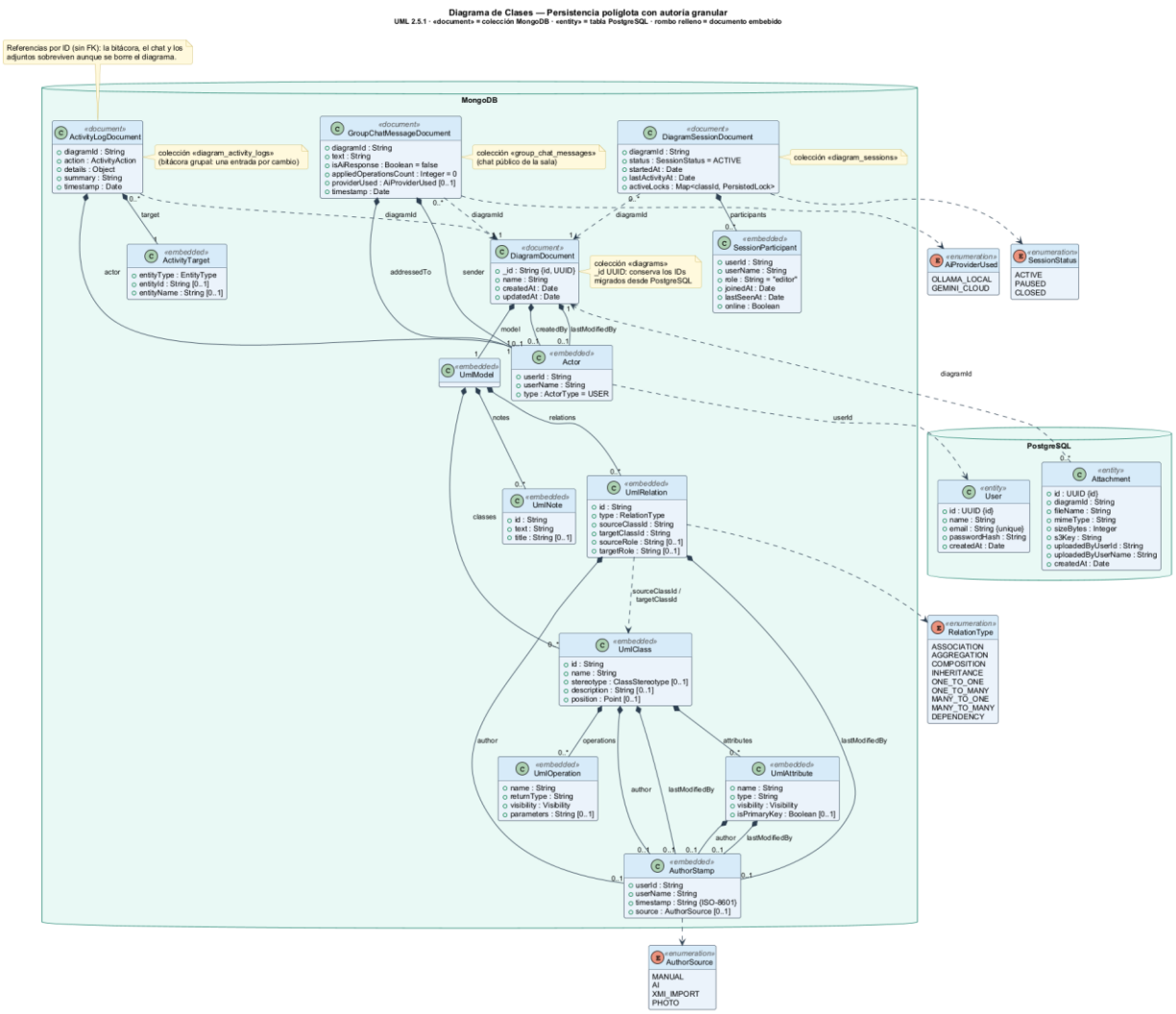


Figura E.3. Diagrama de clases — persistencia políglota con autoría granular embebida

Refleja el código en: `backend/src/diagrams/schemas, uml.types.ts, entidades de PostgreSQL`

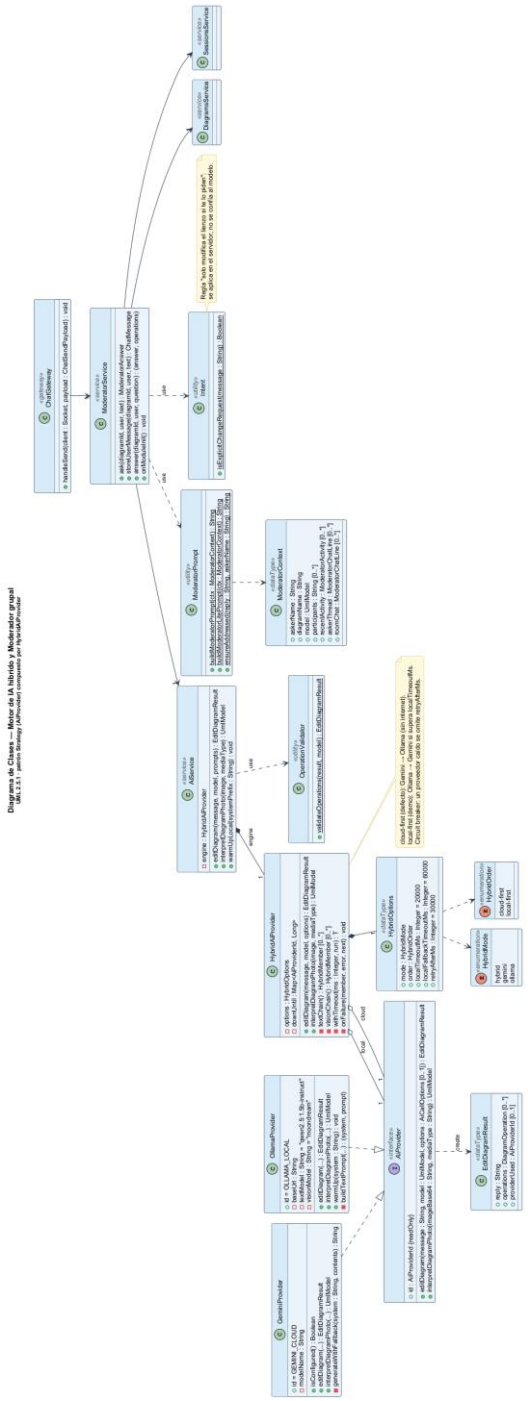


Figura E.4. Diagrama de clases — motor de IA híbrido (patrón Strategy) y Moderador grupal

Refleja el código en: `backend/src/ai/**`

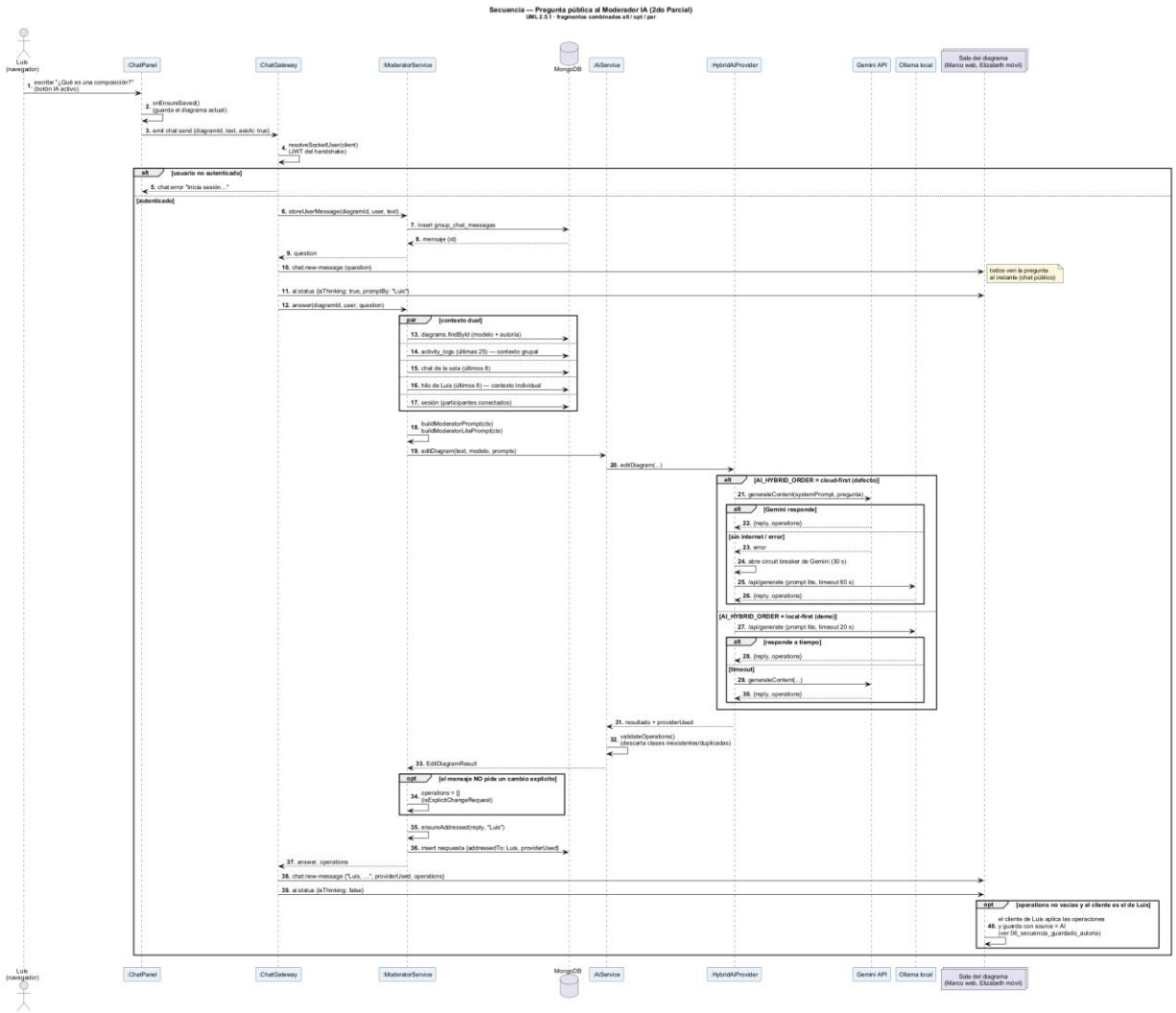


Figura E.5. Diagrama de secuencia — pregunta pública al Moderador IA (alt / opt / par)

Refleja el código en: `ChatPanel.tsx, chat.gateway.ts, moderator.service.ts, hybrid.provider.ts`

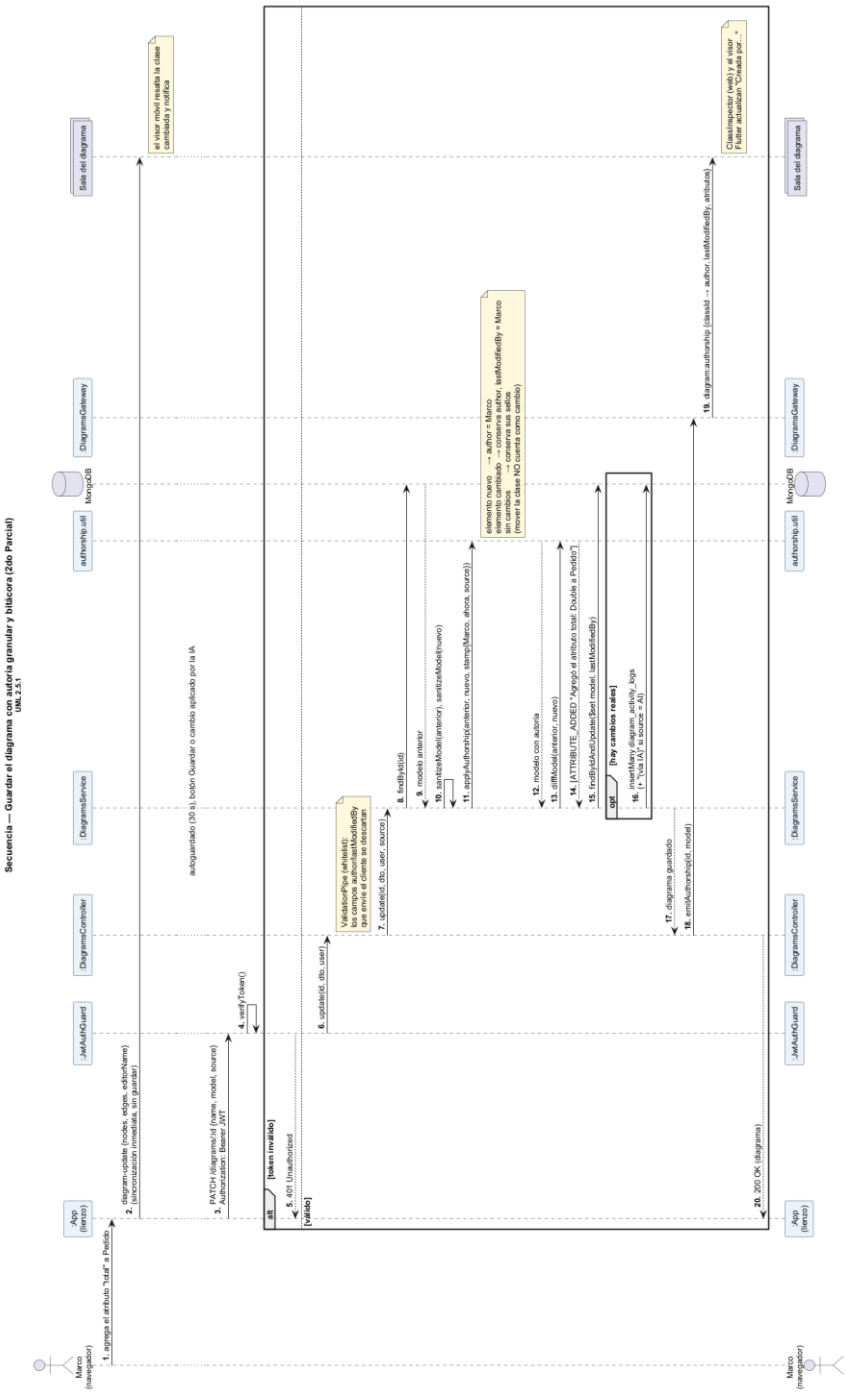


Figura E.6. Diagrama de secuencia — guardado con autoría granular y bitácora

Refleja el código en: `App.tsx (saveDiagram)`, `diagrams.controller.ts`, `diagrams.service.ts`, `authorship.util.ts`

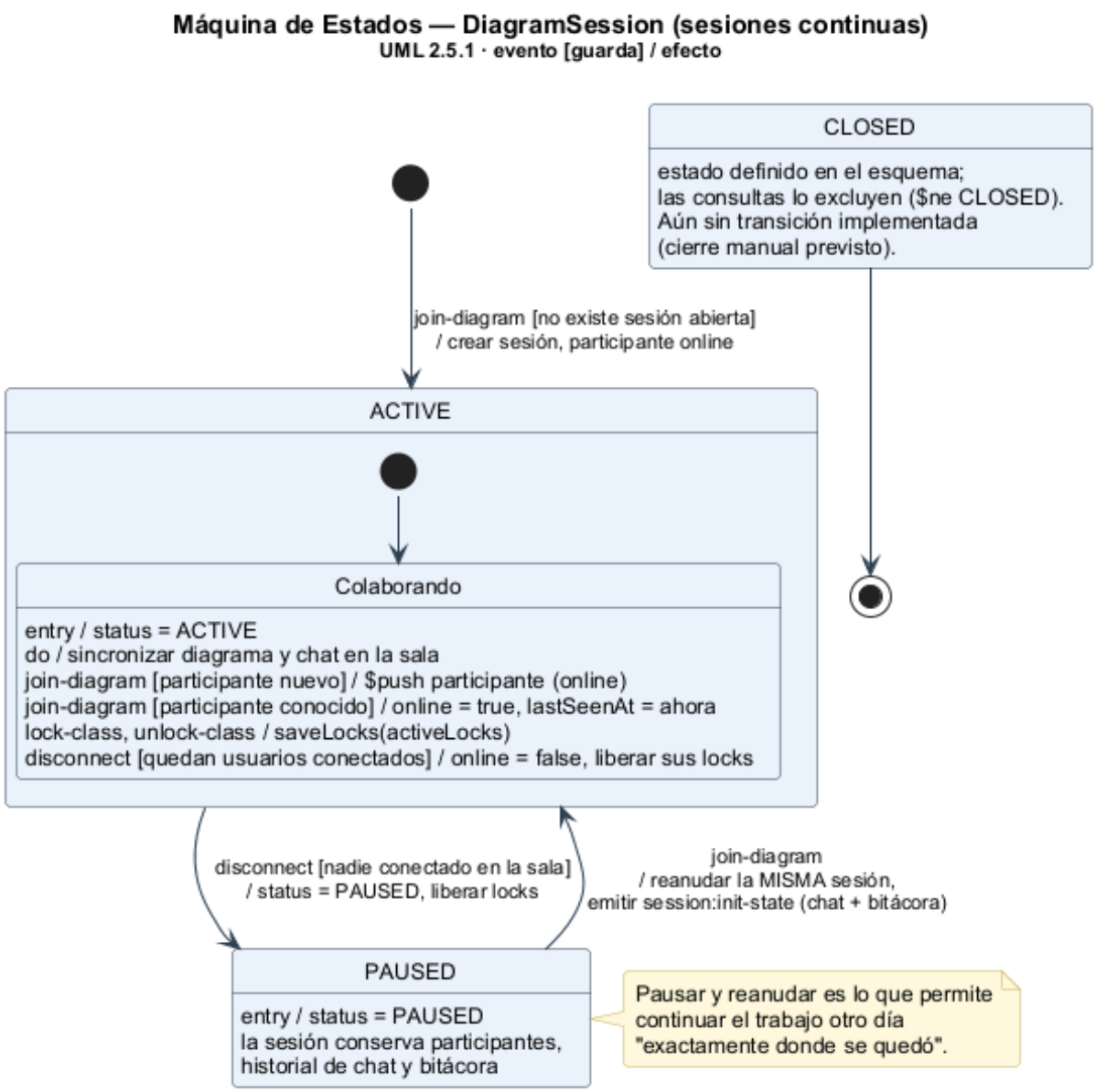


Figura E.7. Máquina de estados — sesión continua (ACTIVE ⇄ PAUSED)

Refleja el código en: `sessions.service.ts, diagrams.gateway.ts`

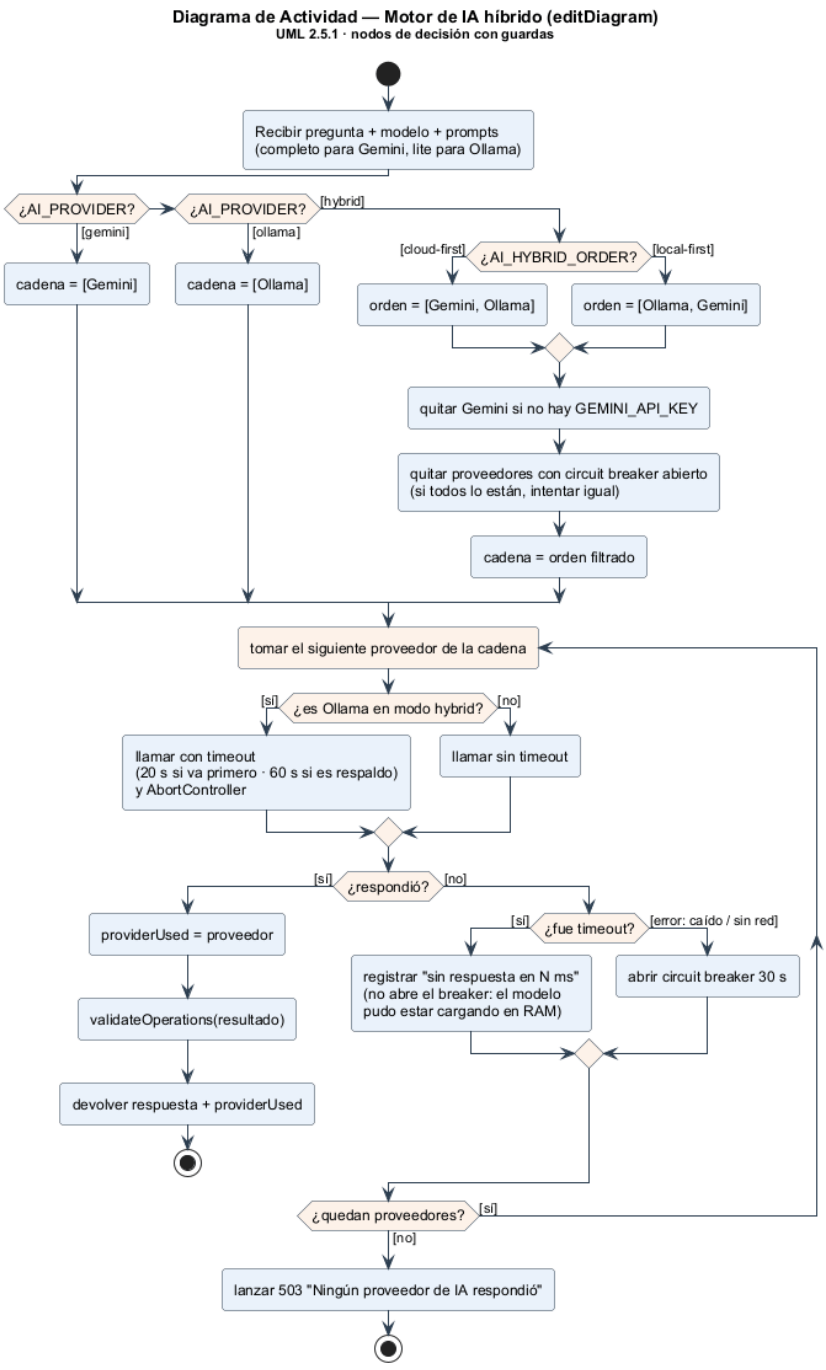


Figura E.8. Diagrama de actividad — decisión del motor de IA híbrido

Refleja el código en: `hybrid.provider.ts, ai.service.ts`

Anexo F. Bibliografía (APA 7)

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Google. (s.f.). *Gemini API documentation*. Google AI for Developers. https://ai.google.dev/gemini-api/docs

Jacobson, I., Booch, G., & Rumbaugh, J. (1999). *The unified software development process*. Addison-Wesley.

MongoDB, Inc. (s.f.). *MongoDB manual*. https://www.mongodb.com/docs/manual/

NestJS. (s.f.). *NestJS documentation*. https://docs.nestjs.com/

Object Management Group. (2005). *MOF 2.0/XMI mapping specification, version 2.1*. https://www.omg.org/spec/XMI/

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

Ollama. (s.f.). *Ollama documentation*. https://github.com/ollama/ollama/tree/main/docs

Rumbaugh, J., Blaha, M., Premerlani, W., Eddy, F., & Lorensen, W. (1991). *Object-oriented modeling and design*. Prentice Hall.

Sadalage, P. J., & Fowler, M. (2012). *NoSQL distilled: A brief guide to the emerging world of polyglot persistence*. Addison-Wesley.

Socket.IO. (s.f.). *Socket.IO documentation*. https://socket.io/docs/v4/

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.